# Belief Revision Assignment Report

A Small Course-Aligned Belief Revision Engine with Optional Mastermind

Group members: *fill in before submission*

Spring 2026

## 1 Introduction

Belief revision is the problem of changing an agent's information state when new input may conflict with what is already believed. In the AGM picture this is analyzed through three operations: expansion, contraction, and revision. The course material treated these ideas mainly on belief sets, but the assignment asks for an implementable *belief base*. We therefore keep a finite set of formulas as the stored state, and compute logical consequences only when needed.

The implementation was written with one design rule in mind: stay as close as possible to the material from the course, while keeping the code short and transparent. The main engine follows the assignment stages directly: propositional formulas in symbolic form, CNF conversion, resolution-based entailment, remainder-based contraction with a priority order, non-closing expansion, and revision via the Levi identity. The optional Mastermind part is deliberately simple and uses a finite possible-world view: each legal code is treated as a candidate world, and feedback removes impossible worlds.

**Main idea.** The project is intentionally small rather than optimized. The goal is to make every logical step visible in the code: parsing, CNF conversion, resolution, remainder computation, selection by priority, and revision by $B * \varphi = (B \div \neg\varphi) + \varphi$.

## 2 Stage 1: Belief Base Design

We represent the agent's information state as a finite belief base

$$B = \{(\varphi_1, \pi_1), \ldots, (\varphi_n, \pi_n)\},$$

where each $\varphi_i$ is a propositional formula and each $\pi_i \in \mathbb{N}$ is a priority. Higher priority means that a formula should be harder to give up during contraction.

This choice follows the assignment closely. A belief set is deductively closed, but a belief base is not. That makes the base computationally manageable and also matches the distinction discussed in the course reading: independent stored beliefs belong to the base, while consequences are derived when needed rather than stored explicitly.

The parser supports the symbolic connectives

$$\neg,\ \wedge,\ \vee,\ \rightarrow,\ \leftrightarrow$$

through the ASCII forms `~`, `&`, `|`, `->`, and `<->`. Internally formulas are stored as a small abstract syntax tree. Every formula also has a canonical ASCII rendering so that duplicates can be recognized reliably.

| Module | Purpose |
|---|---|
| `formula.py` | Formula data structures and canonical printing. |
| `parser.py` | Recursive-descent parser for propositional input. |
| `cnf.py` | Elimination of implications, negation normal form, and CNF conversion. |
| `resolution.py` | Resolution refutation for entailment and consistency. |
| `belief_base.py` | Belief base, contraction, expansion, revision, and equivalence checks. |
| `mastermind.py` | Optional Mastermind code-breaker on a finite world space. |

Table 1: Overview of the implementation.

# 3 Stage 2: Logical Entailment by Resolution

Entailment is implemented in the way emphasized in Lecture 10: proof by refutation. For a belief base $B$ and a query $\varphi$, we test

$$B \models \varphi \quad \text{iff} \quad B \cup \{\neg\varphi\} \text{ is unsatisfiable.}$$

This gives a direct route from propositional entailment to satisfiability checking.

### CNF conversion

Before running resolution, formulas are transformed into conjunctive normal form. The conversion uses the standard three-step pipeline:

1. eliminate $\rightarrow$ and $\leftrightarrow$,
2. push negations inward to obtain negation normal form,
3. distribute disjunction over conjunction.

For example,

$$\alpha \rightarrow \beta \equiv \neg\alpha \vee \beta, \qquad \alpha \leftrightarrow \beta \equiv (\neg\alpha \vee \beta) \wedge (\neg\beta \vee \alpha).$$

The final CNF is stored as a list of clauses, and each clause is stored as a set of literals.

### Resolution

Resolution repeatedly combines clauses containing complementary literals. If the empty clause is produced, the clause set is inconsistent. If no new clause can be generated, the search stops and entailment fails. The same procedure is reused for consistency checking of a belief base.

This part of the project stays very close to the course material: the implementation does not call external theorem-proving packages, and the core algorithm is the standard propositional resolution rule from Lecture 10.

# 4 Stage 3: Contraction by Priority-Guided Partial Meet

The central step is contraction. For a target formula $\alpha$, a remainder is an inclusion-maximal subset of the base that does *not* entail $\alpha$. Formally,

$$B \perp \alpha = \{R \subseteq B \mid R \not\models \alpha \text{ and there is no } R' \text{ with } R \subset R' \subseteq B \text{ and } R' \not\models \alpha\}.$$

This is exactly the course idea: keep as much of the original base as possible while removing commitment to the contracted sentence.

Because the base is finite, the code computes remainders directly by enumerating all subsets and then keeping only the inclusion-maximal safe ones. This is exponential in the size of the base, but it is short, exact, and fully transparent for small assignment-scale examples.

To obtain *partial meet* contraction, we still need a selection function. We use the priority order for that purpose. Each remainder is assigned a retained priority profile, for example the tuple counting how many formulas of each priority level it preserves. The selection function keeps all remainders with the best profile. The final contraction result is the intersection of the selected remainders:

$$B \div \alpha = \bigcap \gamma(B \perp \alpha).$$

This keeps the operator close to the course definition while also using the priority order demanded by the assignment.

**Example.** Let

$$B = \{(p, 2),\ (q, 1),\ ((p \wedge q) \to r, 1)\}.$$

Then $B \models r$. The maximal subsets that stop entailing $r$ are

$$\{p, q\}, \quad \{p, (p \wedge q) \to r\}, \quad \{q, (p \wedge q) \to r\}.$$

The last subset keeps only low-priority beliefs, so it is not selected. The first two have the same best priority profile, and their intersection is $\{p\}$. Hence

$$B \div r = \{p\}.$$

This is an important improvement over the previous version of the project. The code now performs an actual priority-guided *partial meet* contraction rather than choosing only a single preferred remainder.

## 5 Stage 4: Expansion and Revision

For belief bases, expansion should be non-closing. We therefore define

$$B + \varphi = B \cup \{\varphi\},$$

up to duplicate removal. In the implementation, if the same formula already exists, its stored priority is updated only when the new one is higher.

Revision is then defined by the Levi identity,

$$B * \varphi = (B \div \neg\varphi) + \varphi.$$

Since we work with a belief base rather than a belief set, this is the natural internal revision route: first remove enough support for $\neg\varphi$, then add $\varphi$ explicitly. Newly accepted information receives priority $\max(B) + 1$ by default, so the new input can defeat older conflicting support.

## 6 AGM-Inspired Tests

The assignment explicitly asks for tests of Success, Inclusion, Vacuity, Consistency, and Extensionality. Since our data structure is a finite belief base rather than a deductively closed theory, the tests are formulated at the base level but checked through entailment when needed.

| **Postulate** | **Test used in the code** |
|---|---|
| Success | After revision by $\varphi$, the resulting base entails $\varphi$. |
| Inclusion | Every stored formula in $B * \varphi$ comes from $B$ or the new input. |
| Vacuity | If $B \not\models \neg\varphi$, then revision equals simple expansion. |
| Consistency | If $\varphi$ is consistent, then $B * \varphi$ is consistent. |
| Extensionality | Revision by equivalent inputs, e.g. $q$ and $\neg\neg q$, gives equivalent bases. |

There is also a direct contraction test showing that the new operator really performs partial meet with priorities. In total the submission contains 11 unit tests, all of which pass.

# 7 Optional Part: Mastermind

The optional task asks the belief revision engine to play the traditional Mastermind game. To keep this extension as short and course-aligned as possible, we use the possible-world reading of belief change from the course reading: each legal code is treated as a possible world.

For the standard setting with 4 pegs and 6 colors, the initial belief state is the set of all legal codes. The code-breaker starts with a fixed first guess (the first legal code unless the user supplies another one). After each guess, the feedback $(b, w)$ is received, where $b$ is the number of correct colors in the correct positions and $w$ is the number of correct colors in the wrong positions. Revision then removes all candidate worlds that would not have produced the same feedback for that guess.

So if $C$ is the current candidate set, $g$ is the current guess, and $f$ is the observed feedback, the revised state is

$$C' = \{c \in C \mid \text{score}(c, g) = f\}.$$

The next guess is simply the first remaining candidate. This is not an optimized Mastermind strategy, but it is faithful to the idea of belief change: feedback rules out impossible worlds and the agent continues with a belief state that is more informative than before.

| Turn | Guess | Feedback | Remaining candidates |
|---|---|---|---|
| 1 | 1111 | (1 black, 0 white) | 500 |
| 2 | 1222 | (2 black, 0 white) | 48 |
| 3 | 1233 | (3 black, 0 white) | 6 |
| 4 | 1234 | (4 black, 0 white) | 6 |

The trace above is produced by the included demo for the secret code 1234. This optional part is intentionally modest: it uses only the background knowledge that a code must be legal, then revises the finite world set with each new piece of feedback.

# 8 What We Learned

The main lesson from the project is that the course definitions become much clearer when one has to implement them. In particular, the distinction between belief sets and belief bases matters a lot computationally. A deductively closed theory is elegant mathematically, but a finite base is much easier to store and manipulate.

A second lesson is that partial meet contraction is simple to define but non-trivial to implement correctly. The notion of remainder sets is clear on paper, yet one still has to decide how the selection function should be represented in code. Using a priority profile gave a short and deterministic solution that still matches the assignment's emphasis on priority orders.

A third lesson is that the optional Mastermind task can be solved without adding much extra machinery if one switches perspective from formulas to worlds. This keeps the implementation small and ties the game back to the possible-world interpretation of belief revision discussed in the course material.

## 9 Conclusion and Further Work

We implemented a complete belief revision engine for finite propositional belief bases, together with the optional Mastermind extension. The core engine covers symbolic propositional input, CNF conversion, resolution-based entailment, priority-guided partial meet contraction, non-closing expansion, and Levi-style revision. The optional part uses the same idea of information change on a finite space of alternatives.

The most obvious next step would be efficiency. Remainder enumeration is exact but exponential, so larger bases would call for a better search procedure or a different contraction formalism such as kernel-based methods. The Mastermind module could also be improved with a stronger choice rule for the next guess. However, for the purposes of this assignment, the current version keeps the formalism close to the lectures and the code as simple as possible.

## References

1. 02180 Intro to AI, Lecture 8–11 course slides and reading material on logical agents, logical inference, and belief revision.
2. 02180 Intro to AI, Belief Revision Assignment, Spring 2026.
3. Sven Ove Hansson, *Logic of Belief Revision*, Stanford Encyclopedia of Philosophy, sections on partial meet contraction, possible worlds, and belief bases.